



Relatório de Inspeção/Revisão dos Requisitos

Gabriel Castro Freitas ^{*1}, Luiz Gustavo França Barreto Silva ^{†1}, Ryan David de Oliveira Silva ^{‡1}, Orientador Pedro Arthur Melo ^{§1*}

¹ Engenharia de Software

Escola de Tecnologias

Universidade Católica do Salvador (UCSAL)

Av. Prof. Pinto de Aguiar, 2589 Pituaçu, CEP: 41740-090

Salvador/BA, Brasil

¹*{gabrielcastro.freitas@ucsal.edu.br, luizgustavofranca.silva, ryan.silva}@ucsal.edu.br*

^{1*}*{pedroam.nascimento}@pro.ucsal.edu.br*

Setembro 2026

*gabrielcastro.freitas@ucsal.edu.br

†luizgustavofranca.silva@ucsal.edu.br

‡ryan.silva@ucsal.edu.br

§pedroam.nascimento@pro.ucsal.br

1 Introdução

Este relatório busca inspecionar e apontar as ambiguidades e contradições existentes nas regras de negócio do *Projeto Simulador de Batalha - PokeSal - v2*. E, propor novos requisitos refinados, sendo os mesmo, sucintos, detalhados e claros.

Metodologia: cada RA/RF/RNF foi comparado com o enunciado original (quando aplicável) e com sua representação nos diagramas.

2 Tabela de Rastreabilidade

ID	Descrição	Status	Observação
RA001	6 Held Items cadastrados	OK	Requisito autoral. Define itens seguros fixos no domínio. §3.8.
RA002	Limite de 10 turnos por batalha	Sobreposto	Requisito autoral. Redundante com trecho do RNF007. §3.8.
RA003	Desempate por maior HP no 10º turno	OK	Requisito autoral. Critério de vitória por decisão. §3.8.
RF001	Cadastro de treinadores pelo Admin	Contradição	Conflita com a escolha manual do inicial do enunciado original. §3.1.
RF002	Atributos base dos 6 PokéSal	OK	Consistente entre especificação e modelo de domínio.
RF003	Tipo elemental dos PokéSal	Ambíguo	Redação sugere múltiplos tipos em vez de tipo único. §3.1.
RF004	3 Terrenos de batalha cadastrados	Contradição	Afirma que a batalha contém os 3, contrariando o RNF004. §3.3.
RF005	3 Itens de batalha cadastrados	Ambíguo	Confunde tipos cadastrados no sistema com itens do treinador. §3.5.
RF006	Matriz elemental de vantagens	Omisso	Não define multiplicador para confrontos do mesmo elemento. §3.2.
RF007	Efeitos de terreno no combate	Ambíguo	Mistura modificadores de dano e cura no final do turno. §3.3.
RF008	Aplicação de efeitos de status	Omisso	Não especifica durações nem valores exatos de penalidade. §3.7.
RF009	Fórmula de cálculo de dano	Incompleto	Omite o poder do golpe (poder) e mistura operadores. §3.6.
RF010	Limite de 2 itens por batalha	OK	Coerente com o controle de mochila na classe Treinador.
RF011	Consumo de turno ao usar item	OK	Impede a execução de ataque no mesmo turno.
RF012	Ordem de ataque por SPD	Ambíguo	Não explicita a execução do ataque do Pokémon mais lento. §3.4.
RNF001-005	Sorteios e associações aleatórias	OK	Regras preparatórias consistentes entre si.

ID	Descrição	Status	Observação
RNF006	Condições para início do torneio	Omisso	Pressupõe torneio sem detalhar chaveamento ou partidas. §3.7.
RNF007	Encerramento por $HP \leq 0$ ou turnos	Redundante	Sobreposição parcial com os requisitos autorais RA002/RA003. §3.8.
RNF008-010	Diretivas de GenAI, Git e Testes	OK	Diretivas não-funcionais do projeto de engenharia.

Tabela 1 – Rastreabilidade e inspeção estática dos requisitos

3 Achados Detalhados e Propostas de Refinamento

3.1 Seleção de Treinadores e PokéSal

Problema: O enunciado original prevê que o próprio treinador escolha ativamente um PokéSal inicial dentro de um dos trios permitidos (BulbaSal, CharSal, SquirtSal ou ChikoSal, CyndaSal, TotoSal). Em contrapartida, o RF001 estabelece que o Administrador cadastra os treinadores, enquanto o RNF001 determina a associação aleatória de 1 PokéSal entre as 6 opções. Além disso, a redação do RF003 (“cada PokéSal pode ser entre 3 tipos elementais”) gera ambiguidade, podendo dar a entender que um único PokéSal possui múltiplos elementos simultâneos. **Decisão e Refinamento:** Para manter a automação e viabilizar a simulação sem interrupções de I/O de tela, adota-se a atribuição aleatória gerenciada pela classe controladora SimuladorDeBatalha. O RF003 foi reescrito formalmente para garantir a regra de tipo único: “Cada PokéSal possui exatamente um tipo elemental associado, pertencente ao enumeration TipoElemental (‘FOGO’, ‘AGUA’ ou ‘PLANTA’).”.

3.2 Matriz Elemental

Problema: O RF006 especifica rigorosamente as vantagens e desvantagens cruzadas (Fogo vs. Planta, Água vs. Fogo, Planta vs. Água com multiplicadores $2.0\times$ e $0.5\times$), mas é omissivo em relação ao confronto entre PokéSals do mesmo tipo elemental (ex.: Fogo vs. Fogo) ou ataques neutros. **Decisão e Refinamento:** Estabeleceu-se que confrontos de mesmo tipo elemental recebem multiplicador neutro ($1.0\times$). A regra do RF006 passa a vigorar com a cláusula de salvaguarda: “Ataques entre elementos idênticos ou sem relação de vantagem/desvantagem possuem multiplicador neutro igual a 1.0 ”.

3.3 Mecânica e Efeitos de Terreno

Problema 1: O RF004 afirma que “cada batalha deve conter 3 terrenos de batalha cadastrados”. A redação sugere que os três terrenos atuam simultaneamente na mesma partida, o que contradiz o RNF004, que exige o sorteio de um único terreno por confronto. **Problema 2:** O RF007 agrupa sob o termo “efeitos de terreno” modificadores de dano/precisão aplicados no instante do ataque (Asfalto Quente e Poça de Chuva) e regeneração de HP processada ao fim do turno (Canteiro Central). **Decisão e Refinamento:** O RF004 foi corrigido para: “O sistema deve possuir 3 terrenos cadastrados, devendo o SimuladorDeBatalha sortear exatamente 1 terreno ativo para cada

‘Batalha’’. No modelo de classes, os efeitos foram separados: modificadores de ataque alteram o cálculo de dano na classe ‘Batalha’, enquanto restaurações de HP são delegadas ao método de encerramento de turno via interface ‘EfeitoDeStatusTerreno’.

3.4 Turnos, Iniciativa e Ações

Problema (Crítico): O RF012 especifica que “a ordem de ataque do turno é determinada estritamente pelo atributo SPD”. A omissão de como o segundo ataque é processado abriu margem para uma falha grave na primeira versão do código, onde apenas o Pokémon mais rápido agia e o turno terminava. **Decisão e Refinamento:** A execução do turno foi modelada na classe Batalha no método executarTurno(t1, t2, golpe). A regra foi formalizada como: “A cada turno, o combate ordena as ações pelo atributo SPD; o participante de maior SPD executa sua ação primeiro; caso o defensor permaneça com $HP > 0$, este executa sua ação na segunda metade do mesmo turno”. Se o treinador optar por usar um item (RF011), a ação de ataque daquele treinador é substituída pelo consumo do item.

3.5 Gerenciamento da Mochila e Itens

Problema: O RF005 estabelece que “o treinador deve possuir 3 itens de batalha cadastrados”, gerando ambiguidade com o RF010, que limita o uso a no máximo 2 itens por partida. **Decisão e Refinamento:** Esclareceu-se a diferença entre o catálogo de itens do sistema e a mochila do treinador. O RF005 refere-se às três subclasses disponíveis (Potion, SuperPotion, Antidote). A classe Treinador mantém uma lista representando a mochila e um contador - contItensUsados: int, garantindo programmaticamente o bloqueio ao atingir o limite estipulado no RF010.

3.6 Cálculo de Dano e Fórmula Matemática

Problema: A fórmula original apresentada no RF009:

$$\text{Dano} = (\text{ATK} - \text{DEF}) \times \text{Matriz Elemental} \times \text{Efeito Terreno} + \text{Efeito Held Item} + \text{Efeito Status}$$

apresentava falhas graves de engenharia de software e domínio: Omite completamente o atributo de poder base do golpe (poder) da classe Golpe. Sem ele, golpes fracos e fortes teriam exatamente o mesmo efeito. Não prevê tratamento para limites operacionais quando $\text{DEF} \geq \text{ATK}$, o que resultaria em dano nulo ou cura involuntária. Soma valores absolutos de HeldItem e Status sem padronização de escala. **Decisão e Refinamento:** A fórmula foi reestruturada na classe Batalha no método calcularDano(atacante, defensor, golpe), garantindo um piso mínimo de dano igual a 1:

$$\text{DanoBase} = \max(1, (\text{ATK}_{\text{atacante}} + \text{Poder}_{\text{golpe}} - \text{DEF}_{\text{defensor}}))$$

$$\text{DanoFinal} = \lfloor (\text{DanoBase} \times \text{Mod}_{\text{Matriz}} \times \text{Mod}_{\text{Terreno}}) \rfloor + \text{Bônus}_{\text{HeldItem}} + \text{Penalidade}_{\text{Status}}$$

3.7 Efeitos de Status ao Fim do Turno

Problema: O RF008 cita a aplicação de status (Queimado, Envenenado, Paralisado), mas omite os percentuais de redução e a dinâmica de dano contínuo. **Decisão e Refinamento:** Padronizaram-se os valores numéricos através da hierarquia da interface EfeitoDeStatus: Queimado reduz o ATK

do PokéSal atingido em 15 **Problema:** O RNF006 menciona a execução de um “torneio”, contudo a especificação de requisitos não detalha o formato de chaves ou eliminação. **Decisão e Refinamento:** Para evitar o anti-padrão God Class, separou-se a arquitetura de controle em duas entidades distintas: SimuladorDeBatalha atua como controlador macro (responsável pelo cadastro, sorteio de atributos e pareamento do torneio), enquanto a classe Batalha atua como controlador micro, gerenciando o ciclo de vida de uma partida de 10 turnos entre dois treinadores.

3.8 Requisitos Autorais e Redundâncias

Problema: Os requisitos autorais RA001, RA002 e RA003 definem, respectivamente, os 6 Held Items, o limite de 10 turnos por confronto e o critério de desempate por maior HP restante. Foi identificada uma redundância direta entre o RA002/RA003 e o RNF007. **Decisão e Refinamento:** Mantêm-se os RAs como fonte primária das regras autorais da equipe. O RNF007 foi reescrito de forma enxuta, focando estritamente na condição de término por nocaute: “Uma batalha é encerrada imediatamente quando o HP de um dos PokéSals atinge valor ≤ 0 , ou ao atingir o limite de turnos estipulado no RA002”.

4 Conclusão

A inspeção estática efetuada revelou 2 contradições diretas de especificação, 1 omissão estrutural grave na matemática de cálculo de dano, 4 ambiguidades de redação e 3 sobreposições de requisitos. O alinhamento preventivo dessas inconsistências evitou erros de compilação e falhas lógicas em tempo de execução — como a interrupção indevida de turnos e cálculos de dano inconsistentes —, garantindo a perfeita rastreabilidade entre o documento de requisitos, os Diagramas UML de Caso de Uso e de Classes, e a suíte de testes automatizados com JUnit.